VEXON DEVELOPER HANDBOOK · GUIDE 07

# Testing, Git and Safe Code Changes

Change VEXON without breaking it - Git habits, the real test suites and their results, regression tests and a release checklist

| | |
|---|---|
| **Written for** | A student who changes VEXON (alone or with Claude Code) and must not break existing features |
| **Project** | VEXON / CampusConnect (repository: this folder's parent project) |
| **Evidence** | Generated from a direct inspection of the repository source code |
| **Repository state** | branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s) |
| **Generated** | 16 October 2026, 21:16 |
| **Rebuild** | python docs/developer-handbook/build_handbook.py |

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

# Contents

**Where the repository stands right now** . . . 3
**Git essentials for VEXON** . . . 3
The commands you will use every day . . . 3
Undoing things (from safest to most dangerous) . . . 3
**Inspecting changes made by Claude Code (or anyone)** . . . 4
**The test suites in this repository** . . . 4
**Results from this audit (16 October 2026)** . . . 5
The three server failures, investigated . . . 5
**Running and reading tests** . . . 6
Run everything, or one file . . . 6
Interpreting a failure . . . 6
**Writing a regression test in VEXON style** . . . 7
**Validating role permissions and critical workflows** . . . 8
**Database safety during development** . . . 8
**Development vs production workflow** . . . 8
**Reviewing dependency and security-sensitive changes** . . . 9
**A small change, tested and committed: the full routine** . . . 9
**Pre-release checklist** . . . 9
**Source references** . . . 10

# Where the repository stands right now

> **Warning**
> At the time of this audit, the working tree held **more than 120 uncommitted changes** on branch `fix/csrf-legacy-cookie` (web, server, Kotlin, docs). The `mobile-ios/` folder is entirely untracked. Uncommitted work is one accidental command away from being lost. Worse, several whole modules are **untracked: they have never been committed at all**. They include bus tracking (`server/src/services/transport/`, `server/src/controllers/transportController.js`, `client/src/pages/transport/`, the `Bus*` models), exams (`examController.js`, `Exam.js`), staff early-exit and cover (`staffController.js`), `server/src/middleware/coeGuard.js` (which `routes/index.js` imports), and 9 server test files. A fresh clone of the repository would not even start the server. Committing them is the most urgent safety task. Before starting anything new, review it with `git status` and `git diff`, then commit it in logical pieces on the right branch, or stash it. Never mix it with new changes.

Branches found: `main` (default), `fix/csrf-legacy-cookie` (current), `mobile-parity-and-security-fixes`, and their `origin/` copies. The history has about 83 commits, starting 24 September 2026.

# Git essentials for VEXON

## The commands you will use every day

| Command | What it does | When |
|---|---|---|
| `git status` | Lists changed, staged and untracked files | Before and after every task |
| `git diff` | Shows unstaged line changes | Review before staging |
| `git diff --staged` | Shows what will be committed | Right before `git commit` |
| `git diff --stat` | One line per changed file | Get a quick size of a change |
| `git log --oneline -15` | Recent commits | Find when something changed |
| `git log -p -- server/src/middleware/security.js` | History of one file, with diffs | Understand why code is the way it is |
| `git blame <file>` | Last commit for each line | Find the commit (and message) behind a line |
| `git switch -c fix/short-name` | New branch from the current one | Every new piece of work |
| `git add <file>` | Stage one file | Prefer named files over `git add .` |
| `git commit -m "Fix: ..."` | Record the staged change | Small, single-purpose commits |
| `git stash push -m "wip"` / `git stash pop` | Park and restore uncommitted work | Switching tasks |

## Undoing things (from safest to most dangerous)

| Situation | Command | Risk |
|---|---|---|
| Discard my edits to one file | `git restore <file>` | Loses those uncommitted edits |
| Unstage a file | `git restore --staged <file>` | None |
| Undo a pushed commit | `git revert <commit>` | Safe: adds a new commit that reverses it |
| Fix the last commit message (not pushed) | `git commit --amend` | Rewrites history; do not use after pushing |

| Situation | Command | Risk |
|---|---|---|
| Throw away everything uncommitted | `git reset --hard` | **Destroys all uncommitted work.** Stash first |
| Rewrite pushed history | `git push --force` | **Can destroy teammates' work.** Avoid |

> **Key idea**
> Commit messages in this repository explain *why*: "Do not start the OTP resend cooldown when the SMS fails to send", "Fix 403 CSRF errors from a stale legacy cc_csrf cookie". Keep that style. Your future self reads them with `git log`.

# Inspecting changes made by Claude Code (or anyone)

1. `git status`: which files changed? Is anything unexpected listed, such as `.env`, a lockfile, or files in a module you did not ask about?
2. `git diff --stat`: is the size reasonable for the request?
3. `git diff <file>` for each file. Read every line, not just the summary.
4. Look specifically for these:
   - removed or loosened checks: `authorize(`, `assert...`, `validate`, `pick(`, rate limiters, CSRF;
   - changed role lists in `server/src/constants.js` or `client/src/utils/constants.js`;
   - new dependencies in any `package.json`, or in `mobile-kotlin/gradle/libs.versions.toml`;
   - changed API shapes (URLs, request or response fields) that other clients rely on;
   - deleted or weakened tests;
   - hard-coded URLs, keys, coordinates or test data in production code.
5. Run the tests (next section) **yourself**, and read the summary lines.
6. Only then stage and commit.

VS Code's **Source Control** panel (Ctrl+Shift+G) shows the same diffs side by side, and lets you stage individual lines.

# The test suites in this repository

| Suite | Location | Framework | Command | Kind |
|---|---|---|---|---|
| Server | `server/tests/*.test.js` (80 files) + `helpers.js` | Node's built-in `node:test` + `assert` | `npm test --prefix server` | **Integration**: real Express, Socket.IO and MongoDB (`cmp_test`) |
| Web client | `client/tests/*.test.mjs` (5 files) | `node:test` | `npm test --prefix client` | **Unit**: pure modules (bus geometry, my-location, idle tracker, parent mobile, return location) |
| Expo app | `mobile/tests/*.test.mjs` (4 files) | `node:test` | `npm test --prefix mobile` | Unit |
| Kotlin app | `mobile-kotlin/app/src/test/java/com/vexon/campus/` (8 files) | JUnit 4 | `gradlew.bat testDebugUnitTest` (from `mobile-kotlin/`) | Unit (JVM; no device) |
| iOS | `mobile-ios/Packages/VEXONKit/Tests/`, `mobile-ios/VEXONTests/`, `VEXONUITests/` | Swift Testing / XCTest | `mobile-ios/Scripts/test-kit.sh`, `test-app.sh` (Mac) | Unit and UI smoke; **cannot run on Windows** |

| Suite | Location | Framework | Command | Kind |
|---|---|---|---|---|
| Live smoke | `server/scripts/realtime-smoke.mjs` | Script | `node server/scripts/realtime-smoke.mjs` (server running) | Manual end-to-end check of web and mobile chat |
| CI | `.github/workflows/ci.yml` | GitHub Actions | On every PR and push to `main` | npm audit, server tests (Mongo 7), client build, bundle secret scan |
| Security scans | `secret-scan.yml` (gitleaks), `zap-baseline.yml` (OWASP ZAP, staging only, when `ZAP_STAGING_URL` is set) | GitHub Actions | Automatic | Secrets in history; passive web scan |

> **Evidence status**
>
> **There are no browser end-to-end tests** (no Playwright or Cypress). Full UI flows are verified manually. The server integration tests come closest: they drive the real API and sockets.

# Results from this audit (16 October 2026)

These commands were **actually executed** on the audit machine (Windows 11, Node 24.15, local MongoDB service, Java 24 / Gradle 9.3.1):

| Command | Result |
|---|---|
| `npm test --prefix client` | **Passed**: 46 tests, 0 failed |
| `npx vite build --outDir <scratch folder>` (inside `client/`) | **Passed**: built in about 27 s. Warning: the `BusMap` chunk is over 900 kB (about 1.05 MB minified) |
| `npm test --prefix mobile` | **Passed**: 13 tests, 0 failed |
| `gradlew.bat :app:testDebugUnitTest --rerun` (inside `mobile-kotlin/`) | **Passed**: 50 tests, 0 failures, 0 errors (forced re-execution; the first run reported UP-TO-DATE and was not counted) |
| `npm test --prefix server` | **Failed: 721 tests, 718 passed, 3 failed** (about 34 minutes; npm exit code 1). See the analysis below |
| iOS tests | **Not run**: they need macOS with Xcode |
| APK builds, device tests, real GPS, real SMS, real push | **Not run** in this audit |

The README's claim of "637 at last run" for the server suite is historical. Rely on your own run.

## The three server failures, investigated

| Failing test | Re-run alone | Cause found |
|---|---|---|
| `server/tests/announcement-tampering.test.js` (the whole file failed after about 3 s, before any test ran) | **Passed 5/5** | Intermittent: it failed during file start-up in the long full run, not in a test assertion. Re-run it when it appears; investigate further if it repeats |
| `gate-exit-campus-day.test.js`: "a pass starting today can be used at 2 AM campus time" (expected 200, got 404) | **Fails every time** | A date-dependent test. It fixes the time with `clock.now = () => 2026-10-10 02:00 IST` (`server/src/utils/clock.js`), but `verifyGatePass` first calls `expireGatePasses()`, which uses the real `new Date()`. Once the real date passed the test pass's validity (after 10/11 October 2026), the sweep expires the pass, so the code is not found (404) |
| Same file: "a pass starting tomorrow still cannot be used tonight" (expected `false`, got `undefined`) | **Fails every time** | Same cause: the pass is swept as expired before the check under test runs |

> **Warning**
> This is a perfect example for Guide 06. The failure is real and repeatable, but the evidence points at **mixed clocks** (`clock.now` in one place, `new Date()` in another), not at broken gate-pass logic in production. A fix belongs either in the test (dates relative to the real "now") or in the code (`expireGatePasses` reading `clock.now()`). Decide which, write it down, and keep the test. Deleting a failing test hides the problem. **No application code was changed during this audit.**

# Running and reading tests

## Run everything, or one file

```bash
npm test --prefix server                       # all server tests (needs MongoDB running locally)
cd server && node --test tests/sos.test.js     # one file
cd server && node --test --test-name-pattern="non-responders" tests/sos.test.js   # matching tests only
npm test --prefix client
cd mobile-kotlin && gradlew.bat testDebugUnitTest
```

The server tests use `TEST_MONGO_URI` or `mongodb://127.0.0.1:27017/cmp_test`. `server/tests/helpers.js` **refuses** any database name not ending in `_test`, and drops the test database before and after each file. Your development data is never touched. The server runs with `--test-concurrency=1`, so files run one at a time against the shared test database. For the same reason, **never start two test runs at once**: each file drops `cmp_test`, which would break the other run.

> **Note**
> During this audit every test file logged `[db] MongoDB connected (atlas) -> 127.0.0.1:27017/cmp_test`. The word "atlas" comes from `DB_TARGET=atlas` in the local `server/.env`, which `env.js` uses for the label. The tests still connected to the **local** `cmp_test` database, because `helpers.js` passes that URI explicitly. The host part of the line is the truth. The label is only misleading.

## Interpreting a failure

A failing `node:test` looks like this (**hypothetical** example):

```
? Campus SOS: non-responders get 403 on list, ack, resolve (35.2ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:
  200 !== 403
      at TestContext.<anonymous> (file:///.../server/tests/sos.test.js:38:10)
```

Read it in this order: **the test name** (which behaviour broke), **expected vs actual** (403 expected, 200 returned, so a permission check is missing), **the file and line of the assertion** (open it to see the request). Then trace that route (Guide 03). A real failure is information about the code. Never "fix" it by changing the expected value unless the requirement truly changed.

# Writing a regression test in VEXON style

Tests create their own users and data through helpers. They never use real accounts. The pattern, from `server/tests/sos.test.js`:

```js
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { startServer } from './helpers.js';

let ctx;
before(async () => { ctx = await startServer(); });
after(async () => { await ctx.stop(); });
```

A new test (a **hypothetical** example: clubs currently have no direct tests):

```js
test('student join request is approved by a club admin', async () => {
  const owner = await ctx.createUser({ role: 'club_admin' });
  const student = await ctx.createUser();
  const club = await ctx.models.Club.create({
    name: 'Test Club', slug: 'test-club', description: 'A club used only by this test.', category:
    'technical',
    admins: [owner._id], members: [owner._id], status: 'approved', createdBy: owner._id,
  });
  const s = await ctx.loginWeb(student);
  const o = await ctx.loginWeb(owner);

  const join = await ctx.request('POST', `/clubs/${club._id}/join`, { token: s.token, body: { message:
    'hi' } });
  assert.equal(join.status, 200);

  const approve = await ctx.request('POST', `/clubs/${club._id}/requests/${student._id}/approve`, {
    token: o.token });
  assert.equal(approve.status, 200);

  const fresh = await ctx.models.Club.findById(club._id).lean();
  assert.ok(fresh.members.map(String).includes(String(student._id)));
});
```

> **Note**
> This example was written from the route list and model fields. Before relying on it, check `server/src/models/Club.js` (required fields, `status` values) and `clubController.js` (exact status codes). Adjust the expected values to what the code *should* do, then make sure it passes.

Good regression tests:

- reproduce the **exact** bug (the role, data and request that failed);
- assert the **status code and the important field**, not every byte;
- include the **negative** case (the wrong role gets 403, another department gets 403 or 404);
- use synthetic data only.

# Validating role permissions and critical workflows

Permissions are where VEXON's most important tests live (`roles`, `security-audit`, `security-qa-audit`, `defense-in-depth`, `college-wide-roles`). When you change anything role-related, write a small **matrix test**: one request, many roles, expected statuses.

| Request | student | faculty | hod (same dept) | hod (other dept) | admin |
|---|---|---|---|---|---|
| `POST /api/timetable` | 403 | 403 | 201 | 403/422 | 201 |
| `GET /api/sos` | 403 | 403 | 200 | 200 | 200 |

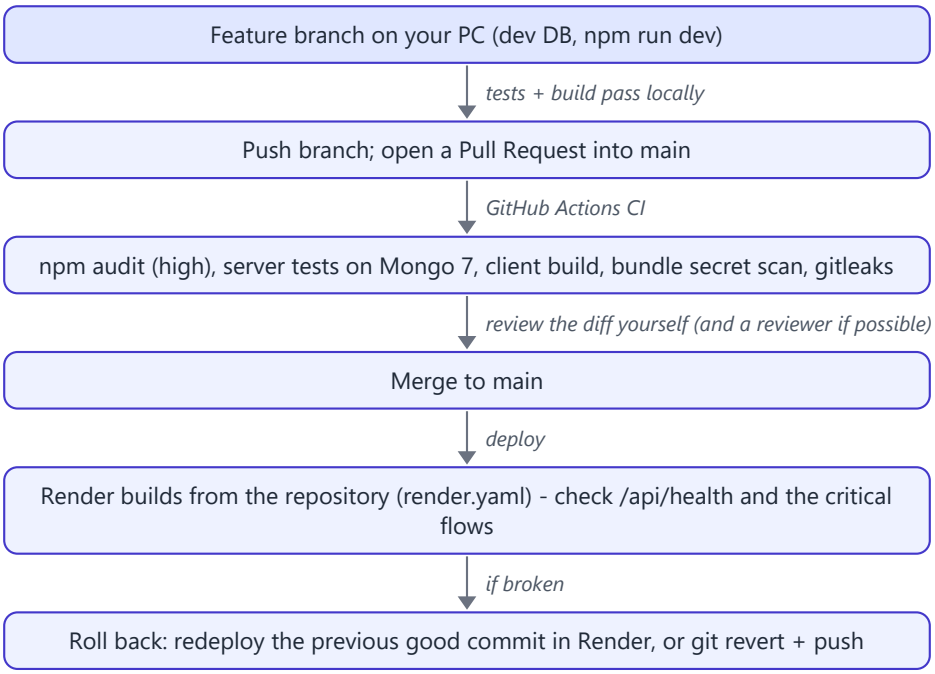
(Fill such a table from `server/src/constants.js` and the route's `authorize(...)` **before** writing the test, then let the test confirm it.)

Critical workflows to re-test manually before any release: sign-in and sign-out on web and Android; marking attendance; the gate-pass chain up to approval and the gate scan; raising and acknowledging an SOS; sending a chat message between two users; the bus map loading.

# Database safety during development

- Your dev database is `campusconnect` (local). Tests use `cmp_test`. **Production is Atlas**: never point your local `.env` at it while developing, and never run `npm run seed` against it (seed **wipes** data).
- Before a risky migration script (`npm run migrate:break-periods --prefix server`), run its **dry run** first. That script only writes with `-- --apply`.
- Take a `mongodump` before any data-changing script, and restore-test it (Guide 04).

# Development vs production workflow




> **Note**
> Confirm in the Render dashboard **which branch** auto-deploys and whether auto-deploy is on. `render.yaml` does not pin a branch, so this cannot be verified from the repository.

# Reviewing dependency and security-sensitive changes

- New package? Ask whether an existing one already does the job. Check its maintenance and licence. The diff must include the **lockfile** change (`package-lock.json`).
- Run `npm audit --omit=dev` in `server/` and `client/`. CI fails on **high** severity.
- After a client build, run `npm run verify:bundle` (`scripts/verify-bundle.mjs`). It fails if the bundle contains anything that looks like a secret or a source map.
- Security-sensitive files deserve a second, slow read: `server/src/middleware/*`, `server/src/utils/tokens.js`, `server/src/utils/sessionIdle.js`, `server/src/constants.js`, `server/src/app.js` (CSP, CORS, trust proxy), `render.yaml`, `.github/workflows/*`.
- `.env` files are ignored by `.gitignore`. If one is ever committed, **rotate the secrets** first. Removing the file in a new commit is not enough (see the comment in `secret-scan.yml`).

# A small change, tested and committed: the full routine

```bash
git status                                  # 1. clean, or understood
git switch -c fix/sos-note-length           # 2. branch
# 3. edit (e.g. a validation rule in server/src/routes/index.js)
# 4. add/adjust a test in server/tests/sos.test.js
cd server && node --test tests/sos.test.js # 5. the focused test
npm test                                    # 6. the full server suite
cd .. && npm run build --prefix client      # 7. if client/ changed
git diff                                    # 8. read every line
git add server/src/routes/index.js server/tests/sos.test.js
git commit -m "Limit SOS note to 300 characters on the server"   # 9. commit with the why
```

# Pre-release checklist

- [ ] Working tree clean; the release is a known commit on `main`.
- [ ] `npm test --prefix server` passes (with the count noted).
- [ ] `npm test --prefix client` passes; `npm run build --prefix client` succeeds.
- [ ] `npm run verify:bundle` passes.
- [ ] Kotlin `testDebugUnitTest` passes if Android changed; APKs rebuilt and installed on a real phone.
- [ ] `npm audit --omit=dev` shows no high-severity issue in `server/` or `client/`.
- [ ] No new secrets, hard-coded URLs or debug flags; `.env.example` updated with any **new variable names**.
- [ ] Production environment variables set on the host for every new setting.
- [ ] Fresh database backup taken **and** a restore tested recently.
- [ ] Manual smoke test of the critical workflows as the affected roles.
- [ ] Rollback plan written down (previous commit id; how to redeploy it).
- [ ] README and this handbook updated if behaviour changed.

## Source references

`server/package.json` · `client/package.json` · `mobile/package.json` · `server/tests/helpers.js` · `server/tests/sos.test.js` · `server/tests/auth.test.js` · `client/tests/` · `mobile-kotlin/app/src/test/java/com/vexon/campus/BusMathTest.kt` · `mobile-ios/Scripts/test-kit.sh` · `server/scripts/realtime-smoke.mjs` · `scripts/verify-bundle.mjs` · `.github/workflows/ci.yml` · `.github/workflows/secret-scan.yml` · `.github/workflows/zap-baseline.yml` · `.gitignore` · `render.yaml` · `server/src/models/Club.js` · `server/src/controllers/clubController.js`